

NITheCS Project

EVALUATING CONFORMAL PREDICTION FOR UNCERTAINTY QUANTIFICATION IN MACHINE LEARNING-BASED TEMPERATURE PREDICTION

Khethiwe Ntshangase

```
In [ ]: %pip install pandas numpy requests matplotlib seaborn scikit-learn statsmodels xgbo
```

1. Import Libraries

```
In [31]: import pandas as pd
import numpy as np
import requests
import matplotlib.pyplot as plt
import seaborn as sns

from statsmodels.tsa.seasonal import STL

from sklearn.ensemble import RandomForestRegressor, VotingRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from xgboost import XGBRegressor

from mapie.regression import SplitConformalRegressor
```

2. Load Dataset

```
In [32]: url = (
    "https://archive-api.open-meteo.com/v1/archive?"
    "latitude=-33.9833&longitude=18.6"
    "&start_date=2015-01-01"
    "&end_date=2025-12-31"
    "&daily=temperature_2m_max,temperature_2m_min,"
    "temperature_2m_mean,precipitation_sum"
    "&timezone=Africa/Johannesburg"
)

data = requests.get(url).json()

df = pd.DataFrame(data["daily"])
df["time"] = pd.to_datetime(df["time"])

pd.set_option('display.max_columns', None)
```

```
pd.set_option('display.width', 1000)

print(df.head())
print(df.tail())
```

	time	temperature_2m_max	temperature_2m_min	temperature_2m_mean	precipitation_sum
0	2015-01-01	24.4	18.6	21.3	0.0
1	2015-01-02	23.5	17.3	20.4	0.0
2	2015-01-03	24.2	15.6	20.5	0.1
3	2015-01-04	23.5	17.7	20.3	1.1
4	2015-01-05	22.2	18.3	20.2	3.7

	time	temperature_2m_max	temperature_2m_min	temperature_2m_mean	precipitation_sum
4013	2025-12-27	27.9	17.0	22.4	0.0
4014	2025-12-28	30.5	16.8	23.7	0.0
4015	2025-12-29	35.6	17.9	27.1	0.0
4016	2025-12-30	30.2	19.1	22.9	0.0
4017	2025-12-31	26.8	18.5	22.0	0.0

```
In [33]: df = df.rename(columns={
    "temperature_2m_max": "TMAX",
    "temperature_2m_min": "TMIN",
    "temperature_2m_mean": "TMEAN",
    "precipitation_sum": "PRCP"})
df.head()
```

```
Out[33]:
```

	time	TMAX	TMIN	TMEAN	PRCP
0	2015-01-01	24.4	18.6	21.3	0.0
1	2015-01-02	23.5	17.3	20.4	0.0
2	2015-01-03	24.2	15.6	20.5	0.1
3	2015-01-04	23.5	17.7	20.3	1.1
4	2015-01-05	22.2	18.3	20.2	3.7

```
In [34]: df = df.sort_values("time")
df = df.reset_index(drop=True)
```

3. Exploratory Data Analysis (EDA)

```
In [35]: # Check for data information
print(df.info())

# Check data shape
print(df.shape)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4018 entries, 0 to 4017
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  -
 0    time    4018 non-null    datetime64[us]
 1   TMAX     4018 non-null    float64
 2   TMIN     4018 non-null    float64
 3   TMEAN    4018 non-null    float64
 4   PRCP     4018 non-null    float64
dtypes: datetime64[us](1), float64(4)
memory usage: 157.1 KB
None
(4018, 5)
```

```
In [36]: # Check for duplicates
print(df.duplicated().sum())

# Check for missing values
print(df.isnull().sum())

# Check for unique values
print(df.nunique())

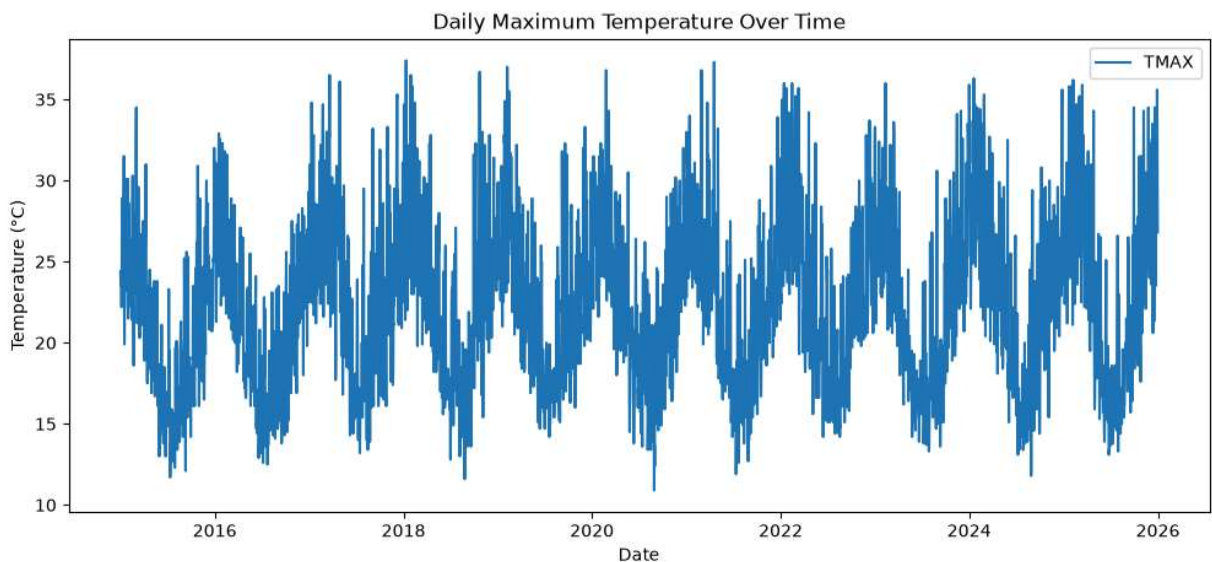
# Check for data types
print(df.dtypes)
```

```
0
time      0
TMAX      0
TMIN      0
TMEAN     0
PRCP      0
dtype: int64
time      4018
TMAX      250
TMIN      200
TMEAN     191
PRCP      204
dtype: int64
time      datetime64[us]
TMAX      float64
TMIN      float64
TMEAN     float64
PRCP      float64
dtype: object
```

```
In [37]: print (df.describe())
```

	time	TMAX	TMIN	TMEAN	PRCP
count	4018	4018.000000	4018.000000	4018.000000	4018.000000
mean	2020-07-01 12:00:00	22.227875	12.884818	17.204032	1.395495
min	2015-01-01 00:00:00	10.900000	0.000000	7.700000	0.000000
25%	2017-10-01 06:00:00	18.100000	10.100000	14.000000	0.000000
50%	2020-07-01 12:00:00	22.050000	13.000000	17.200000	0.000000
75%	2023-04-01 18:00:00	25.600000	15.900000	20.300000	0.500000
max	2025-12-31 00:00:00	37.400000	22.400000	28.000000	80.800000
std	NaN	5.022982	3.860272	3.893224	4.418035

```
In [38]: plt.figure(figsize=(12,5))
plt.plot(df["time"], df["TMAX"], label="TMAX")
plt.title("Daily Maximum Temperature Over Time")
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.show()
```



```
In [39]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Make sure data is sorted by date
df = df.sort_values("time")

# Convert dates to numeric values for trend fitting
x = mdates.date2num(df["time"])
y = df["TMAX"]

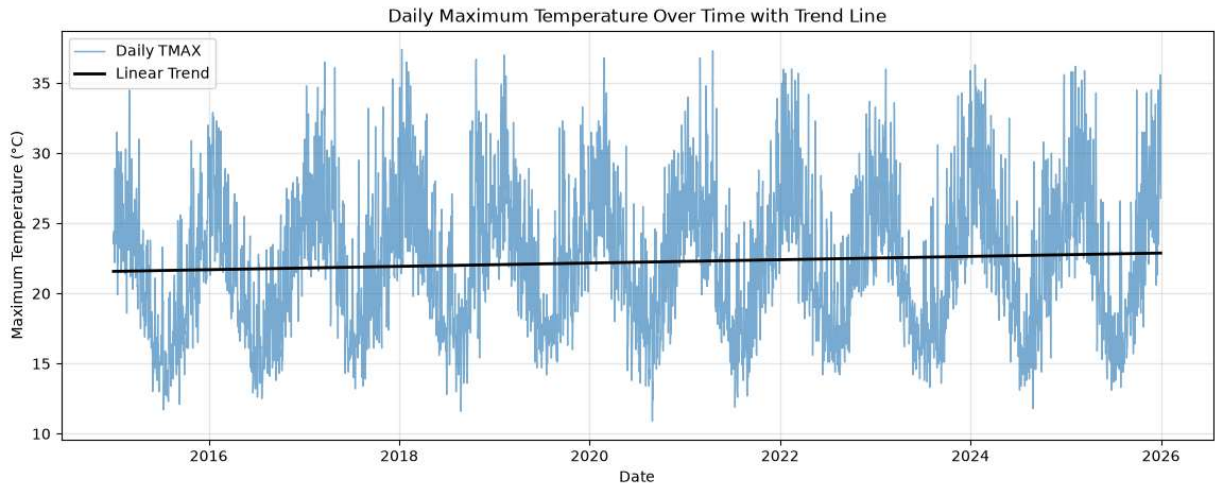
# Fit Linear trend line
trend_coefficients = np.polyfit(x, y, 1)
trend_line = np.poly1d(trend_coefficients)

plt.figure(figsize=(14, 5))

# Daily temperature
plt.plot(df["time"], df["TMAX"], linewidth=1, alpha=0.6, label="Daily TMAX")
```

```
# Trend Line
plt.plot(df["time"], trend_line(x), color="black", linewidth=2, label="Linear Trend")

plt.title("Daily Maximum Temperature Over Time with Trend Line")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



```
In [40]: import matplotlib.pyplot as plt

# Make sure data is sorted by date
df = df.sort_values("time")

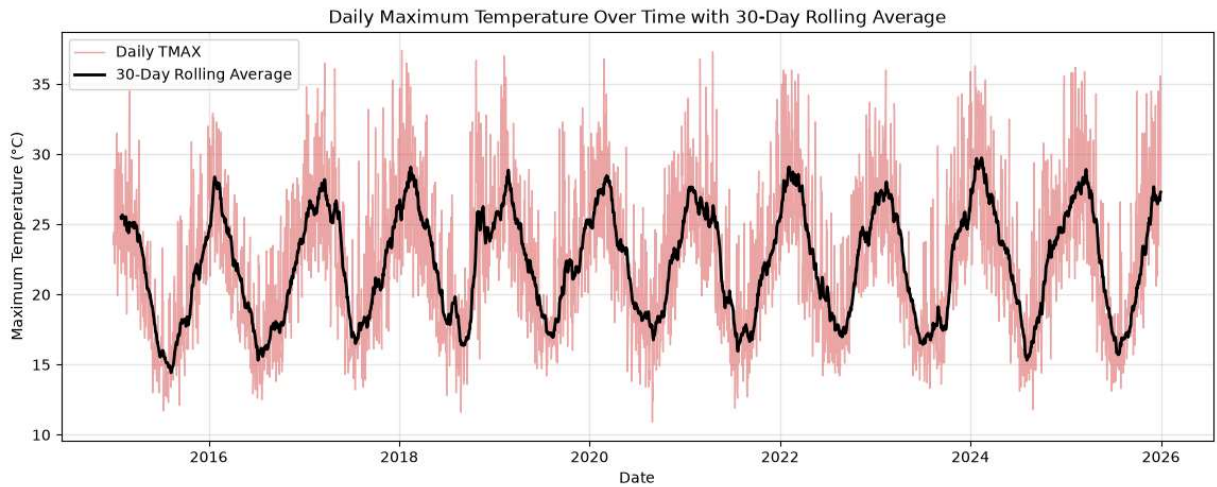
# Create 30-day rolling average
df["TMAX_30day_avg"] = df["TMAX"].rolling(window=30).mean()

plt.figure(figsize=(14, 5))

# Daily temperature
plt.plot(df["time"], df["TMAX"], color="tab:red", linewidth=1, alpha=0.4, label="Da

# Rolling average
plt.plot(df["time"], df["TMAX_30day_avg"], color="black", linewidth=2, label="30-Da

plt.title("Daily Maximum Temperature Over Time with 30-Day Rolling Average")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



```
In [41]: import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates

# Sort data
df = df.sort_values("time")

# Rolling average
df["TMAX_30day_avg"] = df["TMAX"].rolling(window=30).mean()

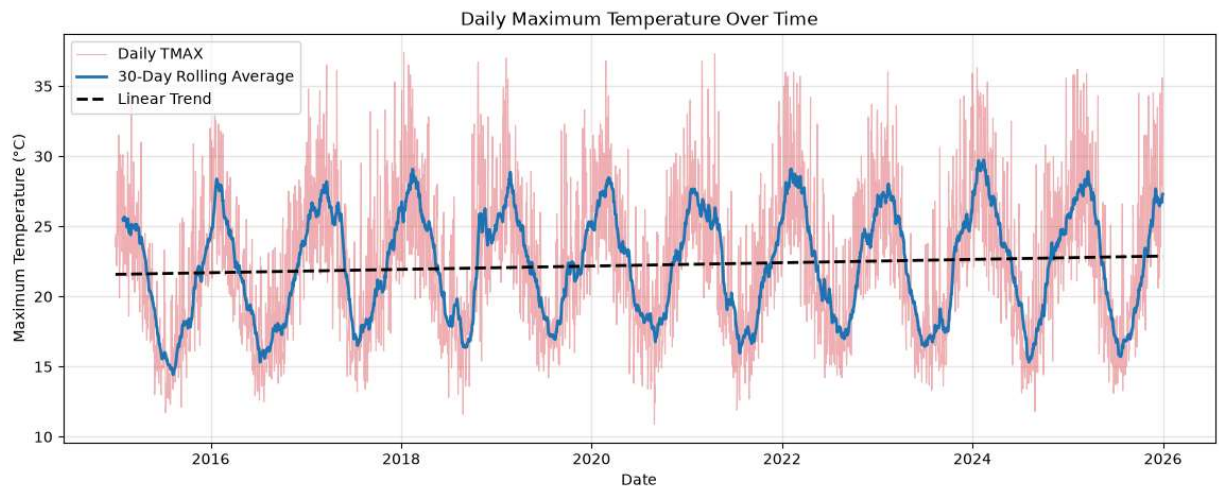
# Linear trend
x = mdates.date2num(df["time"])
y = df["TMAX"]

trend_coefficients = np.polyfit(x, y, 1)
trend_line = np.poly1d(trend_coefficients)

plt.figure(figsize=(14, 5))

plt.plot(df["time"], df["TMAX"], color="tab:red", linewidth=0.8, alpha=0.35, label="Daily TMAX")
plt.plot(df["time"], df["TMAX_30day_avg"], color="tab:blue", linewidth=2, label="30-Day Rolling Average")
plt.plot(df["time"], trend_line(x), color="black", linewidth=2, linestyle="--", label="Linear Trend")

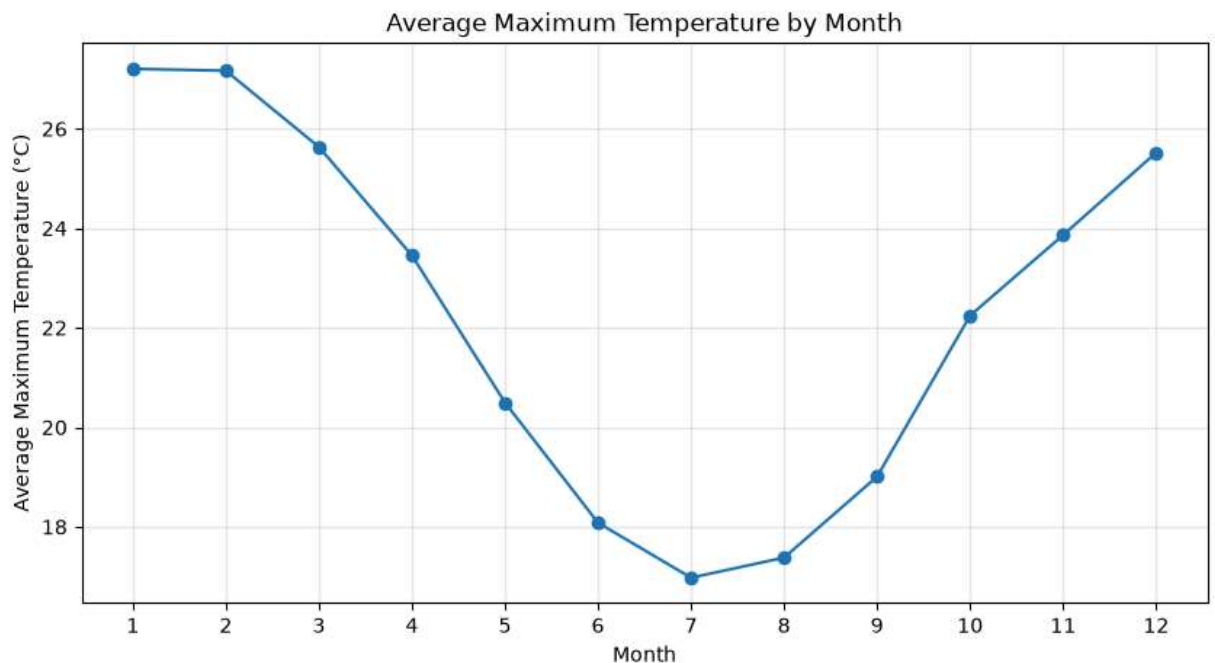
plt.title("Daily Maximum Temperature Over Time")
plt.xlabel("Date")
plt.ylabel("Maximum Temperature (°C)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



```
In [42]: df["month"] = df["time"].dt.month

monthly_avg = df.groupby("month")["TMAX"].mean()

plt.figure(figsize=(10, 5))
plt.plot(monthly_avg.index, monthly_avg.values, marker="o", color="tab:blue")
plt.title("Average Maximum Temperature by Month")
plt.xlabel("Month")
plt.ylabel("Average Maximum Temperature (°C)")
plt.xticks(range(1, 13))
plt.grid(True, alpha=0.3)
plt.show()
```

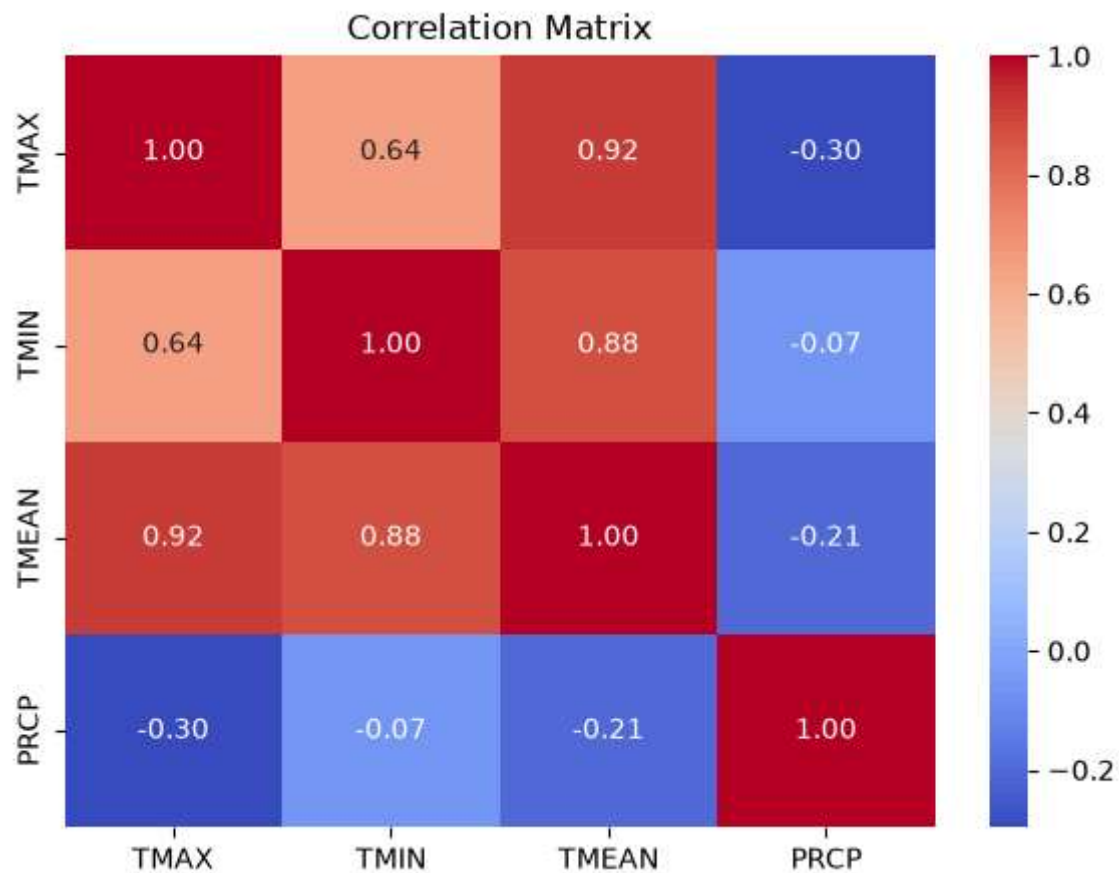


```
In [43]: corr = df[["TMAX", "TMIN", "TMEAN", "PRCP"]].corr()
print(corr)

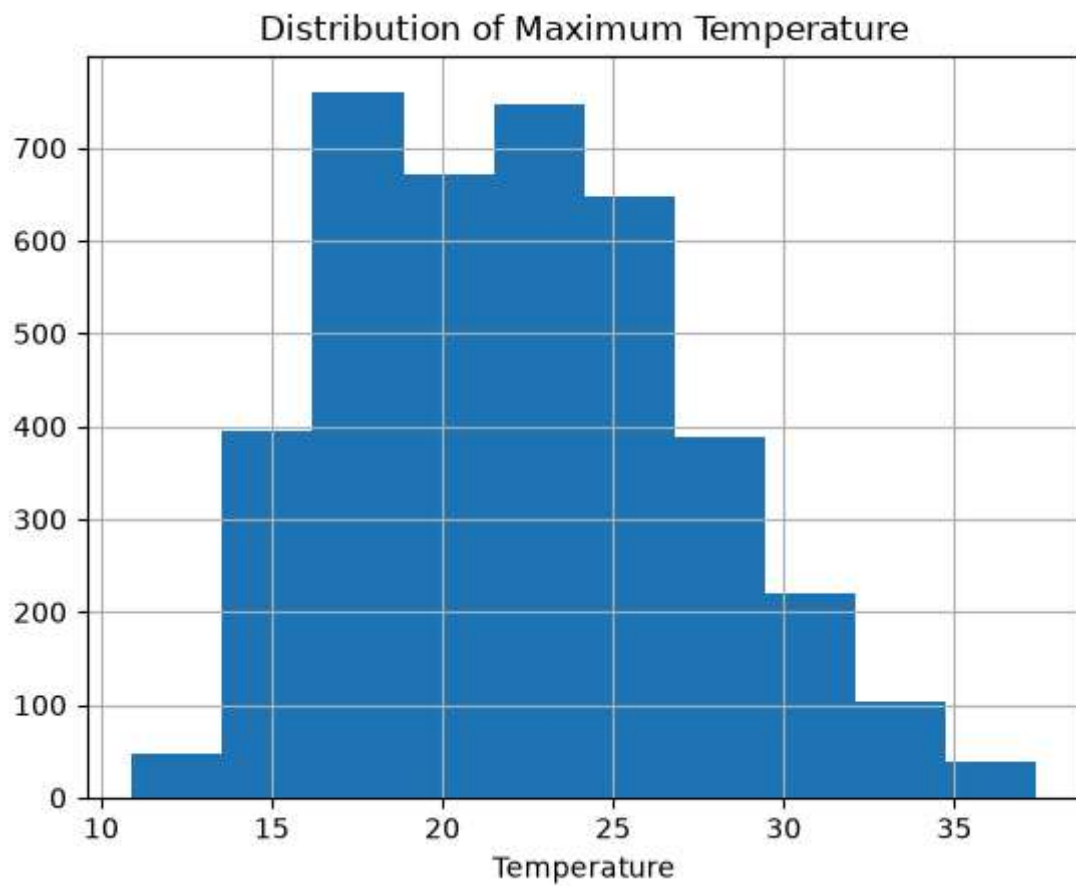
plt.figure(figsize=(7, 5))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
```

```
plt.title("Correlation Matrix")
plt.show()
```

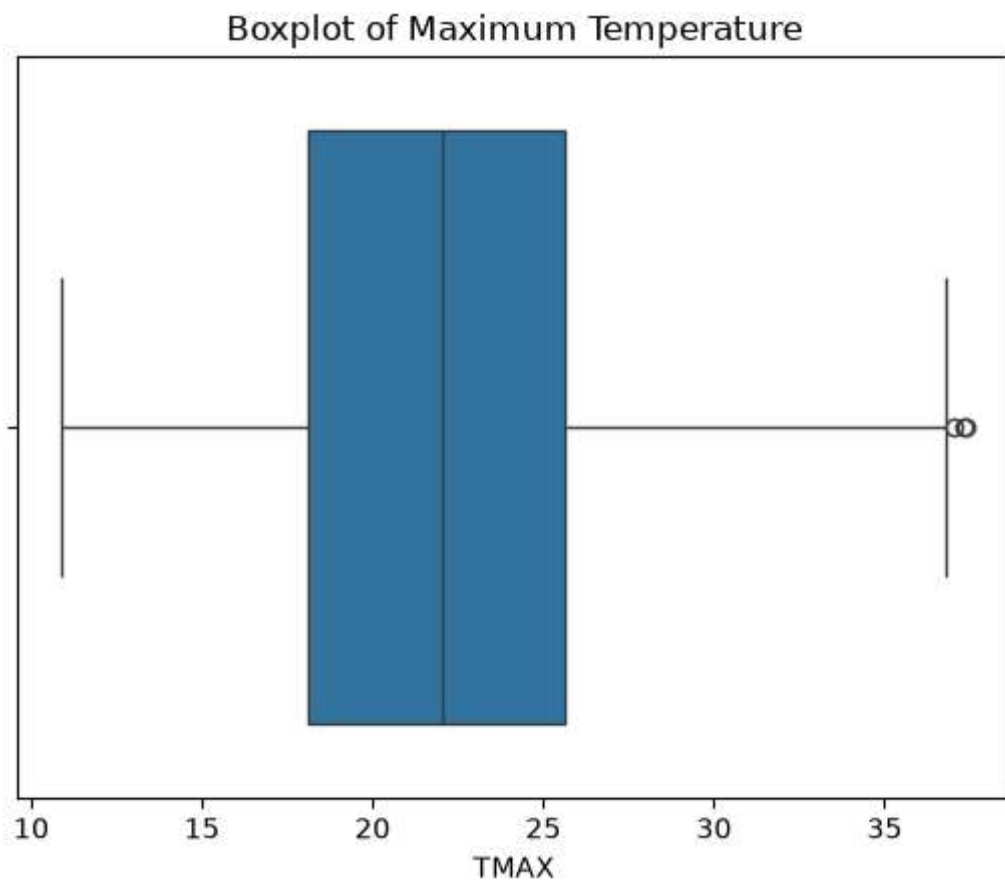
	TMAX	TMIN	TMEAN	PRCP
TMAX	1.000000	0.643191	0.916021	-0.297319
TMIN	0.643191	1.000000	0.877561	-0.065360
TMEAN	0.916021	0.877561	1.000000	-0.205795
PRCP	-0.297319	-0.065360	-0.205795	1.000000



```
In [44]: df["TMAX"].hist()
plt.title("Distribution of Maximum Temperature")
plt.xlabel("Temperature")
plt.show()
```



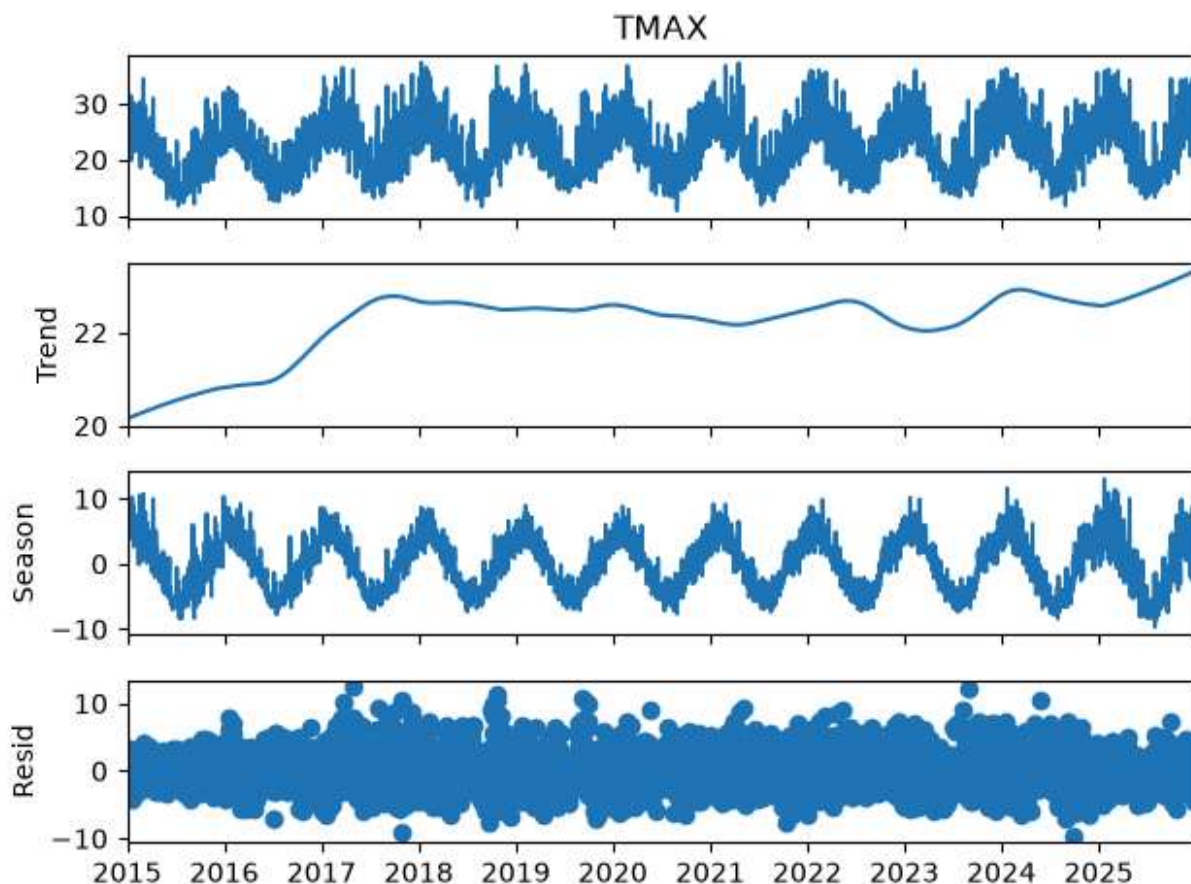
```
In [45]: sns.boxplot(x=df["TMAX"])  
plt.title("Boxplot of Maximum Temperature")  
plt.show()
```



```
In [46]: df_stl = df.set_index("time")

stl = STL(df_stl["TMAX"], period=365)
result = stl.fit()

result.plot()
plt.show()
```



4. Feature Engineering

```
In [47]: df["TMAX_next"] = df["TMAX"].shift(-1)
```

```
In [48]: df["TMAX_lag1"] = df["TMAX"].shift(1)
df["TMAX_lag2"] = df["TMAX"].shift(2)
df["TMAX_lag3"] = df["TMAX"].shift(3)
```

```
In [49]: df["month"] = df["time"].dt.month
df["day"] = df["time"].dt.dayofyear
```

```
In [50]: df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

df["day_sin"] = np.sin(2 * np.pi * df["day"] / 365)
df["day_cos"] = np.cos(2 * np.pi * df["day"] / 365)
```

```
In [51]: df_model = df.dropna().copy()
```

5. Define Features and Target

```
In [52]: features = ["TMIN", "TMEAN", "PRCP", "month", "day", "TMAX_lag1", "TMAX_lag2", "TMA
X = df_model[features]
y = df_model["TMAX_next"]
```

```
In [53]: dates = df_model["time"]
```

6. Split Data

```
In [54]: n = len(df_model)

train_end = int(n * 0.70)
cal_end = int(n * 0.85)

X_train = X.iloc[:train_end]
y_train = y.iloc[:train_end]

X_cal = X.iloc[train_end:cal_end]
y_cal = y.iloc[train_end:cal_end]

X_test = X.iloc[cal_end:]
y_test = y.iloc[cal_end:]

dates_test = dates.iloc[cal_end:]

print("Training size:", len(X_train))
print("Calibration size:", len(X_cal))
print("Test size:", len(X_test))
```

Training size: 2791
Calibration size: 598
Test size: 599

7. Define models

```
In [55]: models = {
    "Random Forest": RandomForestRegressor(
        n_estimators=300,
        random_state=42
    ),

    "XGBoost": XGBRegressor(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
    ),

    "MLP": Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(64, 32),
            activation="relu",
            max_iter=1000,
            random_state=42
        ))
    ])
```

```
    })
}
```

8. Fit all models and evaluate point predictions

```
In [56]: point_results = []
         trained_models = {}

         for name, model in models.items():
             print(f"Training {name}...")

             model.fit(X_train, y_train)
             trained_models[name] = model

             y_pred = model.predict(X_test)

             mae = mean_absolute_error(y_test, y_pred)
             rmse = np.sqrt(mean_squared_error(y_test, y_pred))
             r2 = r2_score(y_test, y_pred)

             point_results.append({
                 "Model": name,
                 "MAE": mae,
                 "RMSE": rmse,
                 "R2": r2
             })

         point_results_df = pd.DataFrame(point_results)
         point_results_df
```

Training Random Forest...

Training XGBoost...

Training MLP...

```
Out[56]:
```

	Model	MAE	RMSE	R2
0	Random Forest	2.409579	3.137446	0.646951
1	XGBoost	2.400336	3.154827	0.643028
2	MLP	2.289811	3.057565	0.664699

8. Apply Conformal Prediction to all three models

```
In [57]: conformal_results = {}
         summary_results = []

         confidence_level = 0.90

         for name, model in trained_models.items():
             print(f"Applying conformal prediction to {name}...")

             cp_model = SplitConformalRegressor(
                 estimator=model,
```

```

        confidence_level=confidence_level,
        prefit=True
    )

    cp_model.conformalize(X_cal, y_cal)

    y_pred, intervals = cp_model.predict_interval(X_test)

    lower = intervals[:, 0, 0]
    upper = intervals[:, 1, 0]

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    coverage = ((y_test >= lower) & (y_test <= upper)).mean()
    avg_width = np.mean(upper - lower)

    summary_results.append({
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2": r2,
        "Coverage": coverage,
        "Average Interval Width": avg_width
    })

    conformal_results[name] = {
        "y_pred": y_pred,
        "lower": lower,
        "upper": upper
    }

results_df = pd.DataFrame(summary_results)
results_df

```

Applying conformal prediction to Random Forest...

c:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not have valid feature names, but RandomForestRegressor was fitted with feature names
warnings.warn(

Applying conformal prediction to XGBoost...

Applying conformal prediction to MLP...

Out[57]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.409579	3.137446	0.646951	0.888147	10.174667
1	XGBoost	2.400336	3.154827	0.643028	0.904841	10.576679
2	MLP	2.289811	3.057565	0.664699	0.889816	9.916161

9. Interpret model comparison table

In [58]: results_df.sort_values(by="MAE")

Out[58]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.289811	3.057565	0.664699	0.889816	9.916161
1	XGBoost	2.400336	3.154827	0.643028	0.904841	10.576679
0	Random Forest	2.409579	3.137446	0.646951	0.888147	10.174667

In [59]: `results_df.sort_values(by="RMSE")`

Out[59]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.289811	3.057565	0.664699	0.889816	9.916161
0	Random Forest	2.409579	3.137446	0.646951	0.888147	10.174667
1	XGBoost	2.400336	3.154827	0.643028	0.904841	10.576679

In [60]: `results_df.sort_values(by="Average Interval Width")`

Out[60]:

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
2	MLP	2.289811	3.057565	0.664699	0.889816	9.916161
0	Random Forest	2.409579	3.137446	0.646951	0.888147	10.174667
1	XGBoost	2.400336	3.154827	0.643028	0.904841	10.576679

10. Plot predictions and conformal intervals on (XGBoost)

In [61]:

```

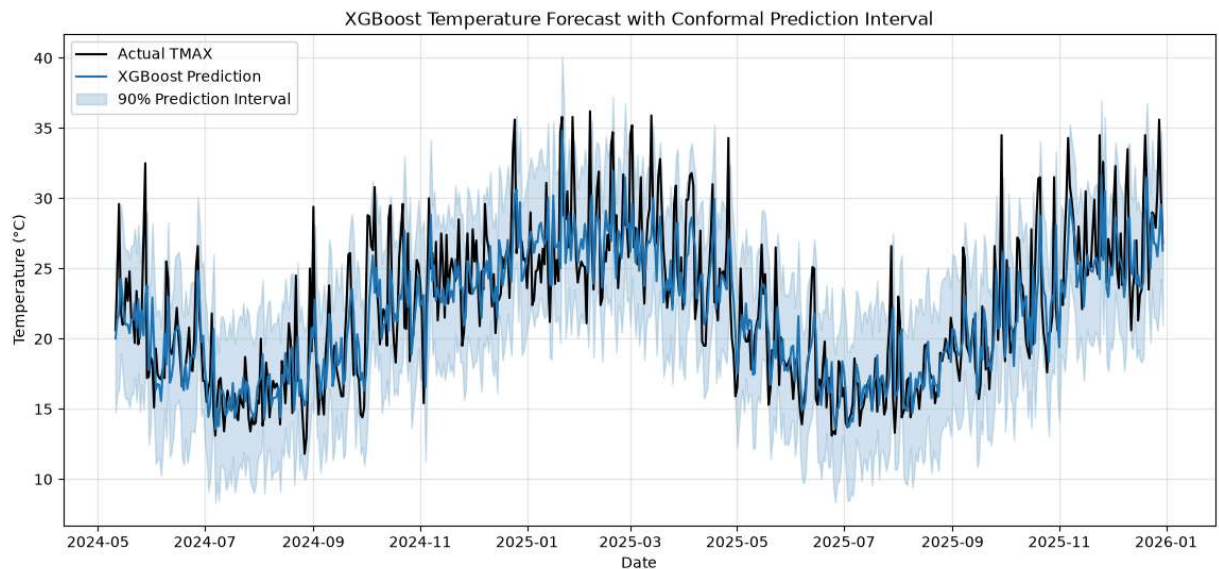
model_name = "XGBoost"

pred = conformal_results[model_name]["y_pred"]
lower = conformal_results[model_name]["lower"]
upper = conformal_results[model_name]["upper"]

plt.figure(figsize=(14, 6))
plt.plot(dates_test, y_test.values, label="Actual TMAX", color="black")
plt.plot(dates_test, pred, label=f"{model_name} Prediction", color="tab:blue")
plt.fill_between(dates_test, lower, upper, color="tab:blue", alpha=0.2, label="90%")

plt.title(f"{model_name} Temperature Forecast with Conformal Prediction Interval")
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

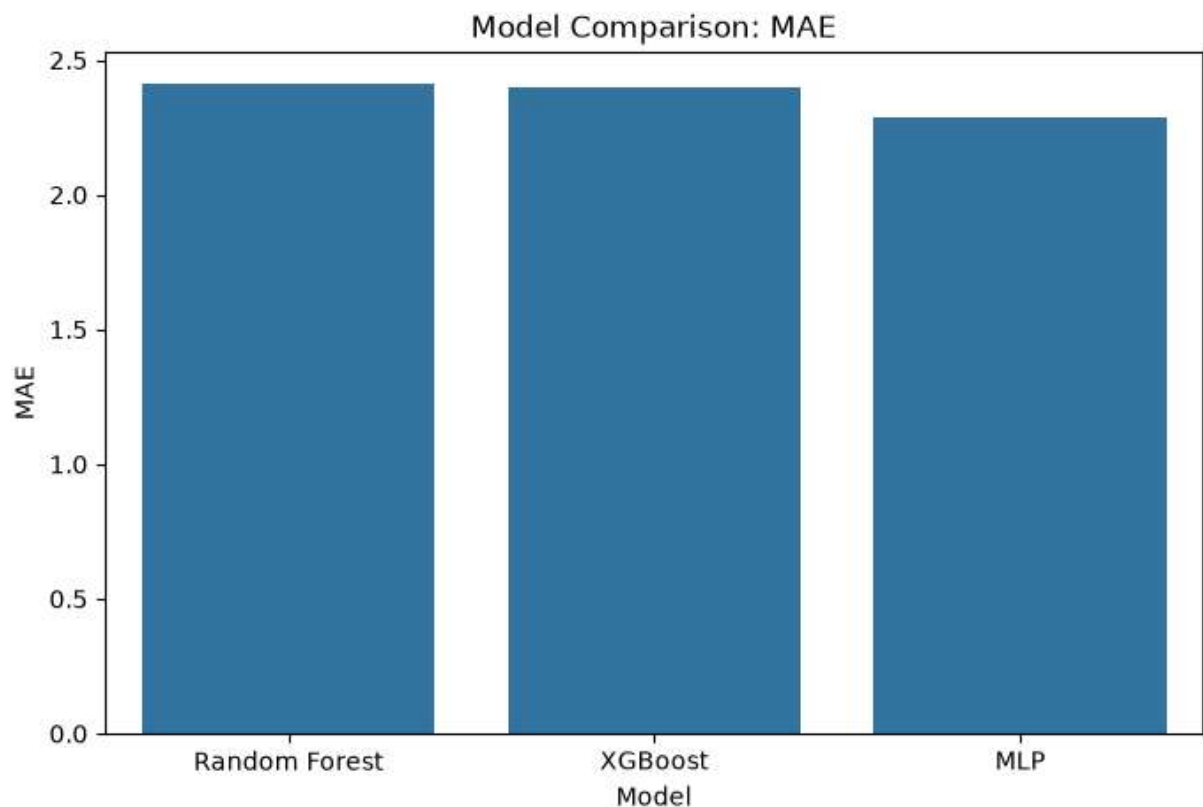
```



11. Plot all model results as bar charts

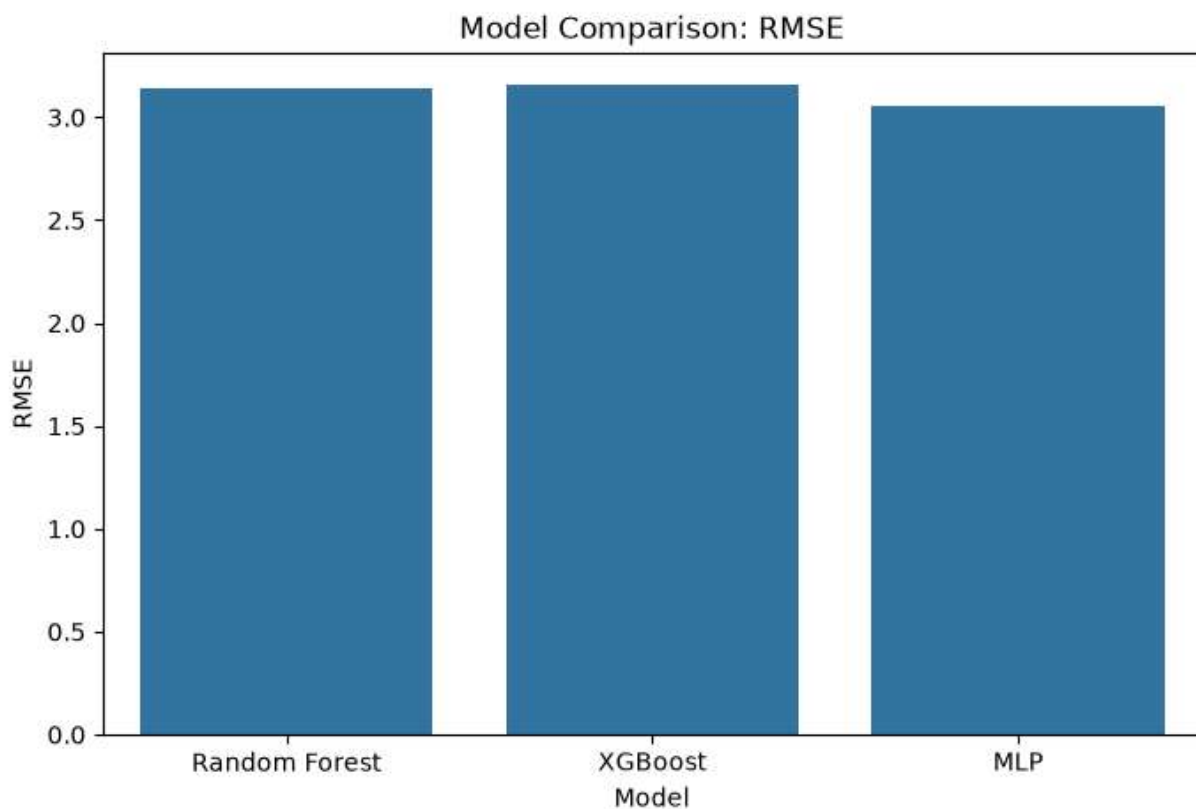
MAE comparison

```
In [62]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="MAE")
plt.title("Model Comparison: MAE")
plt.ylabel("MAE")
plt.show()
```



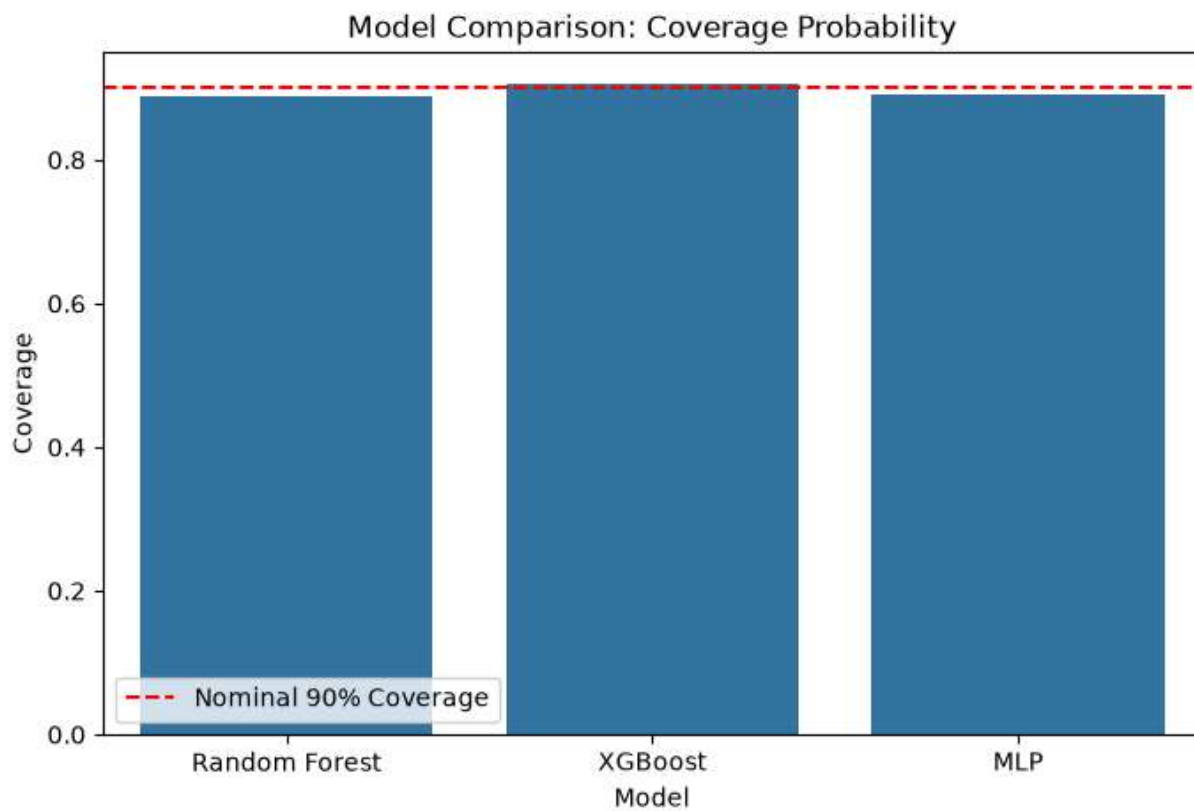
RMSE comparison

```
In [63]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="RMSE")
plt.title("Model Comparison: RMSE")
plt.ylabel("RMSE")
plt.show()
```



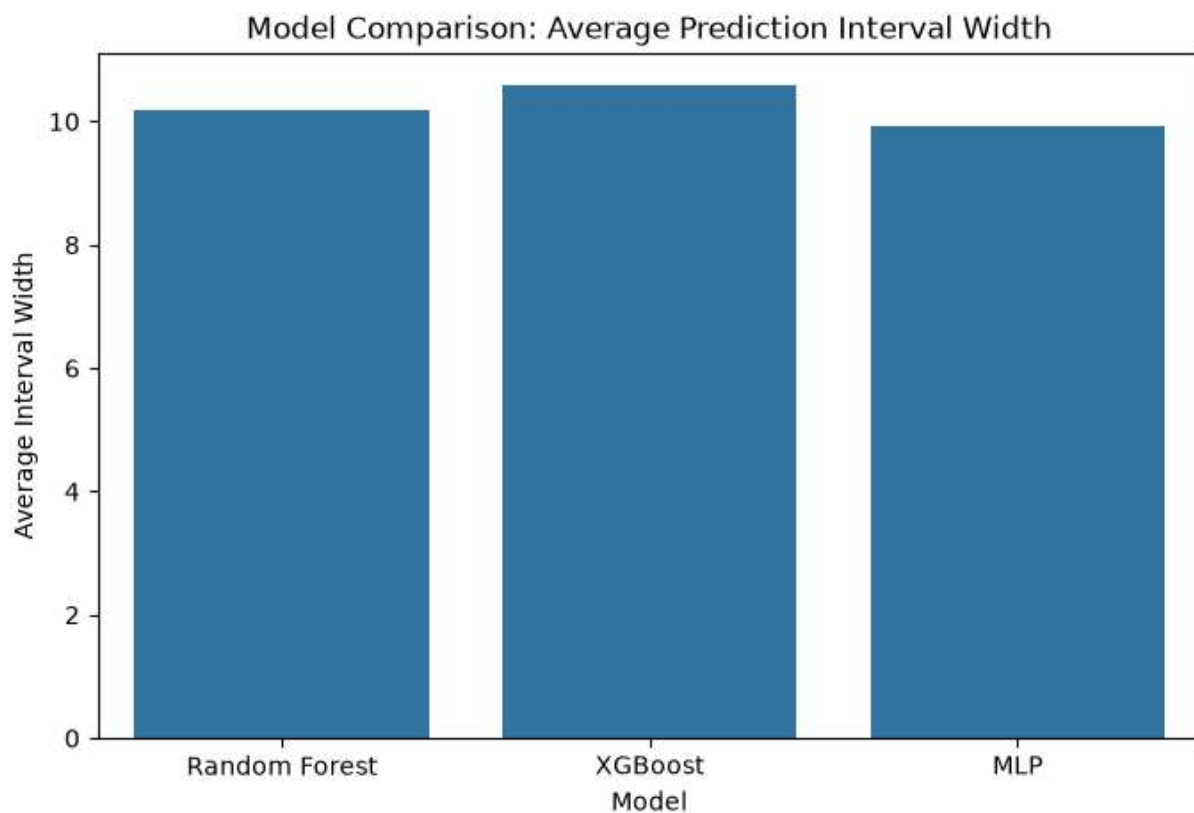
Coverage comparison

```
In [64]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="Coverage")
plt.axhline(0.90, color="red", linestyle="--", label="Nominal 90% Coverage")
plt.title("Model Comparison: Coverage Probability")
plt.ylabel("Coverage")
plt.legend()
plt.show()
```



Interval width comparison

```
In [65]: plt.figure(figsize=(8, 5))
sns.barplot(data=results_df, x="Model", y="Average Interval Width")
plt.title("Model Comparison: Average Prediction Interval Width")
plt.ylabel("Average Interval Width")
plt.show()
```



12. Ensemble model

```
In [66]: ensemble_model = VotingRegressor([
    ("rf", RandomForestRegressor(n_estimators=300, random_state=42)),
    ("xgb", XGBRegressor(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42
    )),
    ("mlp", Pipeline([
        ("scaler", StandardScaler()),
        ("mlp", MLPRegressor(
            hidden_layer_sizes=(64, 32),
            max_iter=1000,
            random_state=42
        ))
    ]))
])

ensemble_model.fit(X_train, y_train)

cp_ensemble = SplitConformalRegressor(
    estimator=ensemble_model,
    confidence_level=0.90,
    prefit=True
)
```

```

cp_ensemble.conformalize(X_cal, y_cal)

y_pred_ens, intervals_ens = cp_ensemble.predict_interval(X_test)

lower_ens = intervals_ens[:, 0, 0]
upper_ens = intervals_ens[:, 1, 0]

ensemble_metrics = {
    "Model": "Ensemble",
    "MAE": mean_absolute_error(y_test, y_pred_ens),
    "RMSE": np.sqrt(mean_squared_error(y_test, y_pred_ens)),
    "R2": r2_score(y_test, y_pred_ens),
    "Coverage": ((y_test >= lower_ens) & (y_test <= upper_ens)).mean(),
    "Average Interval Width": np.mean(upper_ens - lower_ens)
}

ensemble_metrics

```

```

c:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\util
s\validation.py:2827: UserWarning: X does not have valid feature names, but RandomFo
restRegressor was fitted with feature names
  warnings.warn(
c:\Users\user\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages\sklearn\util
s\validation.py:2827: UserWarning: X does not have valid feature names, but Standard
Scaler was fitted with feature names
  warnings.warn(

```

```

Out[66]: {'Model': 'Ensemble',
'MAE': 2.3282746854836303,
'RMSE': np.float64(3.078584684915711),
'R2': 0.6600735078700195,
'Coverage': np.float64(0.8898163606010017),
'Average Interval Width': np.float64(9.968243884004295)}

```

```

In [67]: results_df = pd.concat(
    [results_df, pd.DataFrame([ensemble_metrics])],
    ignore_index=True
)

results_df

```

```

Out[67]:

```

	Model	MAE	RMSE	R2	Coverage	Average Interval Width
0	Random Forest	2.409579	3.137446	0.646951	0.888147	10.174667
1	XGBoost	2.400336	3.154827	0.643028	0.904841	10.576679
2	MLP	2.289811	3.057565	0.664699	0.889816	9.916161
3	Ensemble	2.328275	3.078585	0.660074	0.889816	9.968244

13. Save results

```

In [68]: results_df.to_csv("model_comparison_results.csv", index=False)

```

```
In [69]: df_model.to_csv("processed_temperature_data.csv", index=False)
```